

FinWealth Advisors
Wealth Analytics Data Platform
Prepared By : Helly Patel
INT_1062

1. Project Overview

FinWealth Advisors needed a centralised data warehouse on Microsoft Azure that pulls together data from four operational systems, validates it, transforms it into a clean analytical model, and makes it available for reporting in Power BI.

Component	What it is	Why we needed it
Azure Blob Storage	Cloud file storage	Drop zone for CSV files before ADF picks them up
Azure SQL Database	Cloud relational database	Hosts all staging and DWH tables
Azure Data Factory	ETL orchestration tool	Automates copying, validating, and loading data
Staging Layer	Raw copy of source data	Buffer between source files and the DWH
DWH Star Schema	Dim + fact tables	Optimised structure for analytics and Power BI
SQL Stored Procedures	Dimension loading logic	Handle SCD Type 1, NULL fixing, derived columns
ADF Mapping Data Flows	Fact table loading	Visual transformations for high-volume fact tables
Power BI Views	Pre-joined SQL views	Ready-to-use datasets for dashboards

1.1 Source Data

We received 11 CSV files representing different parts of the business:

CSV File	Staging Table	Row Count	What it contains
Clients.csv	stg_clients	8,000	Client details, segments, risk profiles
Advisors.csv	stg_advisors	350	Advisor info, regions, experience
Accounts.csv	stg_accounts	12,000	Account types, currencies, statuses
Securities.csv	stg_securities	3,000	Asset classes, sectors, tickers
Trades.csv	stg_trades	100,400 (100,000 after dedup)	Buy/sell transactions
Holdings.csv	stg_holdings	96,027	Daily portfolio positions
Cashflows.csv	stg_cashflows	30,000	Deposits, withdrawals, dividends
Fees.csv	stg_fees	18,000	Advisory, management, transaction fees
Prices.csv	stg_prices	301,500	Daily closing prices per security
Benchmarks.csv	stg_benchmarks	4,020	Benchmark index values over time
Performance.csv	stg_performance	110,000	Monthly account returns vs benchmarks

2. Azure Setup

2.1 Resource Group

Setting	Value
Resource Group	RG-FinWealthAnalytics
Region	Southeast Asia

2.2 Storage Account

Setting	Value
Storage Account	finwealthsa
Container	raw
Files uploaded	All 11 CSV files in the raw container

2.3 Azure SQL Database

Setting	Value
SQL Server	finwealth-sql-server
Database	FinWealthDW
Query Tool	Query Editor (preview) in Azure Portal

2.4 Azure Data Factory

Setting	Value
Data Factory	finwealthanalyticsadf
Studio	ADF Studio (Launch Studio)

3. Database Setup — Staging Tables

Every staging table has the same pattern — all source columns plus three audit columns at the end:

Audit Column	Type	Purpose
stg_load_date	DATETIME DEFAULT GETDATE()	When this row was loaded
stg_source_file	VARCHAR(200)	Which CSV file it came from
stg_batch_id	VARCHAR(50)	Which pipeline run loaded it

3.1 Staging Tables Created

Table	Key Columns	Source
stg_clients	ClientID, ClientName, Segment, RiskProfile, PrimaryAdvisorID	Clients.csv
stg_advisors	AdvisorID, AdvisorName, Region, JoinDate, ExperienceYears	Advisors.csv
stg_accounts	AccountID, ClientID, AccountType, OpenDate, BaseCurrency, Status	Accounts.csv
stg_securities	SecurityID, Ticker, SecurityName, AssetClass, Sector, Currency	Securities.csv
stg_trades	TradeID, AccountID, SecurityID, TradeDate, SettleDate, Side, Price, Quantity, Amount	Trades.csv
stg_holdings	AccountID, SecurityID, Quantity, ValuationDate, MarketValue	Holdings.csv
stg_cashflows	CashflowID, AccountID, CashflowDate, Type, Amount	Cashflows.csv
stg_fees	FeeID, AccountID, FeeDate, FeeType, Amount	Fees.csv
stg_prices	SecurityID, PriceDate, ClosePrice	Prices.csv
stg_benchmarks	BenchmarkCode, Description, Currency, IndexValueDate, IndexValue	Benchmarks.csv
stg_performance	AccountID, MonthStart, ReturnPct, BenchmarkCode, ExcessReturnPct	Performance.csv
pipeline_audit_log	log_id, pipeline_name, status, rows_loaded, message	System generated
pipeline_error_log	error_id, pipeline_name, error_time, error_msg	System generated

4. Data Warehouse — Star Schema & ER Diagram

4.1 Dimension Tables

Table	Surrogate Key	Rows	Derived Columns
dim_date	date_sk	5,844	day_of_week, month_name, quarter_name, is_weekday, is_month_end
Dim_Advisor	advisor_sk	350	ExperienceBand — Senior/Mid-Level/Junior/Trainee
Dim_Client	client_sk	8,000	ClientTier (Platinum/Gold/Silver/Standard), TenureYears
Dim_Account	account_sk	12,000	CurrencyGroup — Major/Emerging/Other
Dim_Security	security_sk	3,000	AssetClassGroup — Growth/Fixed Income/Passive/Managed/Alternative
Dim_Benchmark	benchmark_sk	6	No derived columns

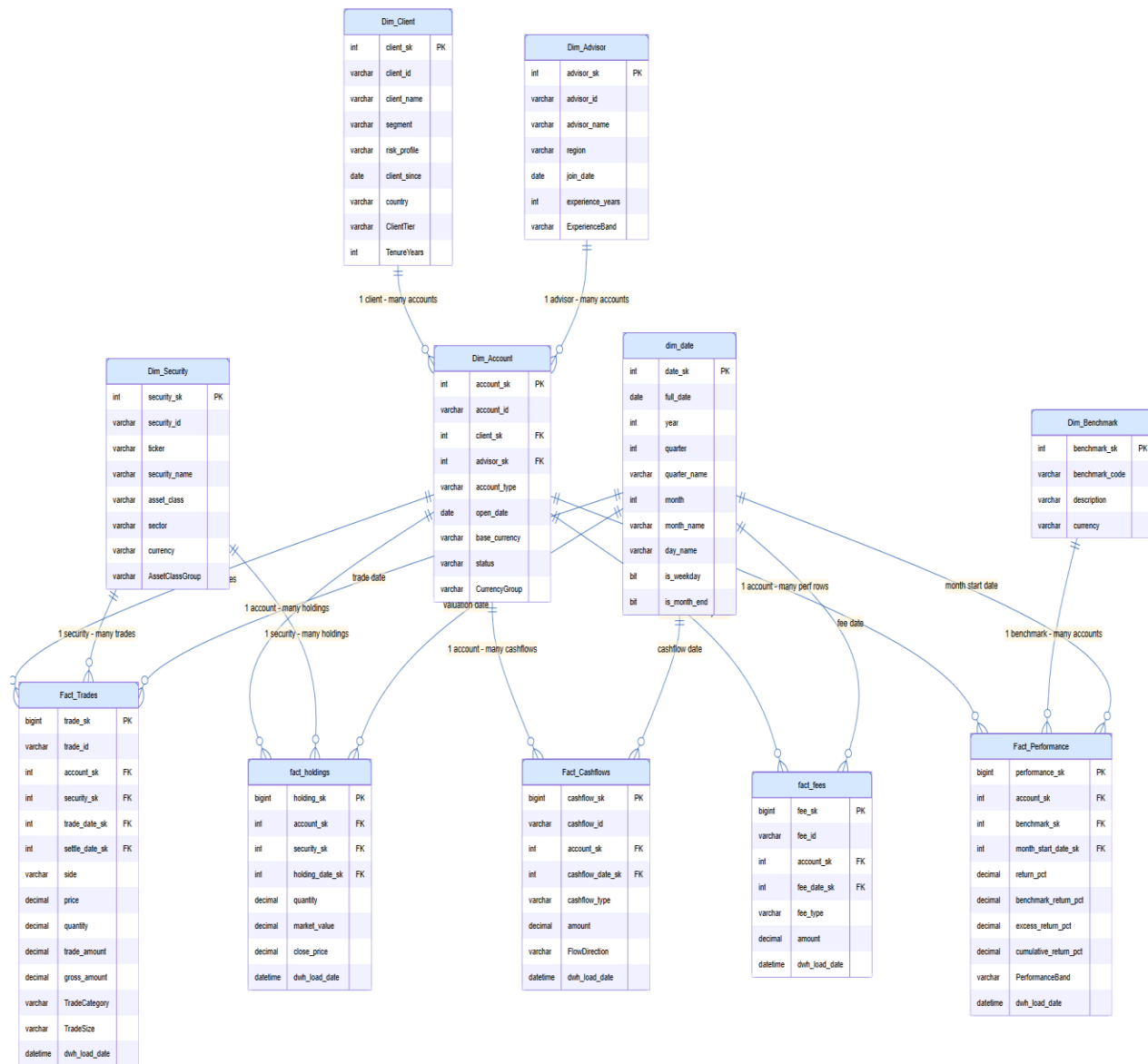
4.2 Fact Tables

Table	Rows	Grain	Derived Columns	Loaded By
fact_holdings	96,027	Account x Security x Date	close_price from stg_prices join	ADF Data Flow
Fact_Trades	100,000	One row per trade	gross_amount, TradeCategory, TradeSize, trade_date_sk, settle_date_sk	ADF Data Flow
Fact_Cashflows	30,000	One row per cashflow	FlowDirection, cashflow_date_sk	ADF Data Flow
fact_fees	18,000	One row per fee	fee_date_sk	ADF Data Flow
Fact_Performance	110,000	Account x Month	PerformanceBand, month_start_date_sk	ADF Data Flow

4.3 SCD Strategy

All dimension tables use SCD Type 1 — when a client changes their segment or an advisor moves region, we overwrite the old value. The `is_current` flag is always 1 for all active records. If the business later needs historical tracking, this can be upgraded to SCD Type 2 by adding `effective_date` and `expiry_date` columns.

4.4 ER Diagram



5. NULL Handling & Business Rules

5.1 NULL Rules

Column Type	Rule	SQL	Why
Text / Name	Replace with Unknown	ISNULL(col, 'Unknown')	Prevents NULLs breaking GROUP BY in Power BI
Date	Replace with 1900-01-01	ISNULL(col, '1900-01-01')	Standard DWH convention for missing dates
Number / Amount	Replace with 0	ISNULL(col, 0)	Safe for SUM and AVG calculations
Key / ID column	Reject the row entirely	WHERE col IS NOT NULL	A NULL key cannot be joined — useless row

5.2 Derived Column Logic

Table	New Column	Logic
Dim_Advisor	ExperienceBand	15+ = Senior. 8-14 = Mid-Level. 3-7 = Junior. <3 = Trainee
Dim_Client	ClientTier	Ultra HNI = Platinum. HNI/HNW = Gold. Mass Affluent = Silver. Else = Standard
Dim_Client	TenureYears	DATEDIFF(YEAR, ClientSince, GETDATE())
Dim_Account	CurrencyGroup	USD/GBP/EUR/JPY = Major. INR/BRL/ZAR/MXN = Emerging. Else = Other
Dim_Security	AssetClassGroup	Equity = Growth. Bond = Fixed Income. ETF = Passive. MF = Managed. Else = Alternative
Fact_Trades	TradeCategory	BUY = Purchase. SELL = Liquidation
Fact_Trades	TradeSize	>=100,000 = Large. >=10,000 = Medium. <10,000 = Small
Fact_Trades	gross_amount	ABS(Amount) — absolute value for SELL trades
Fact_Cashflows	FlowDirection	Deposit/Interest/Dividend = Inflow. Withdrawal = Outflow. Else = Neutral
Fact_Performance	PerformanceBand	>2% = Strong Outperform. >0% = Outperform. 0% = In Line. <0% = Underperform

6. Stored Procedures — Dimension Loading

All dimension transformation logic lives in SQL Stored Procedures called by ADF SP Activities. We chose SPs over Data Flows for dimensions because the business logic (SCD Type 1 merge, surrogate key assignment, derived columns) is easier to test, debug, and version control in SQL.

6.1 Dimension SPs (5 procedures)

Procedure	Loads into	Key logic
usp_load_dim_advisor	Dim_Advisor	Insert new advisors. Update changed ones. Calculate ExperienceBand.
usp_load_dim_client	Dim_Client	Insert new clients. Update changes. Calculate ClientTier and TenureYears.
usp_load_dim_account	Dim_Account	Insert new accounts. Update status changes. Calculate CurrencyGroup.
usp_load_dim_security	Dim_Security	Insert new securities. Update changes. Calculate AssetClassGroup.
usp_load_dim_benchmark	Dim_Benchmark	Insert new benchmark codes only — no updates needed.

6.2 Idempotent Loading

Every SP uses a NOT EXISTS check before inserting. If the pipeline runs twice, no duplicate data is created — the second run finds records already exist and skips them.

7. ADF Mapping Data Flows — Fact Loading

All five fact tables are loaded using ADF Mapping Data Flows. This approach was chosen for fact tables because they involve high row volumes, multi-source joins, and complex derived column logic that benefits from Spark-based parallel processing.

7.1 Why Data Flows for Facts, SPs for Dims?

Approach	Used For	Reason
Stored Procedures	Dimension tables	Complex SCD logic, surrogate key assignment, easier to test in SQL
Mapping Data Flows	Fact tables	High row volumes, multi-source joins, Spark parallel processing

7.2 Data Flows Built (5 total)

Data Flow	Fact Table	Transformations	Canvas Steps
DF_Load_Fact_Trades	Fact_Trades	Filter NULLs, Derive TradeCategory/TradeSize/gross_amount/date_sk, Lookup account_sk + security_sk	Source → Filter → Derived → Lookup x2 → Select → Sink
DF_Load_Fact_Holdings	fact_holdings	Join stg_holdings + stg_prices on SecurityID + Date, Filter NULLs, Derive holding_date_sk, Lookup account_sk + security_sk	Source x2 → Join → Filter → Derived → Lookup x2 → Select → Sink
DF_Load_Fact_Cashflows	Fact_Cashflows	Filter NULLs, Derive cashflow_date_sk + FlowDirection, Lookup account_sk	Source → Filter → Derived → Lookup → Select → Sink
DF_Load_Fact_Fees	fact_fees	Filter NULLs, Derive fee_date_sk, Lookup account_sk	Source → Filter → Derived → Lookup → Select → Sink
DF_Load_Fact_Performance	Fact_Performance	Filter NULLs, Derive month_start_date_sk + PerformanceBand, Lookup account_sk + benchmark_sk	Source → Filter → Derived → Lookup x2 → Select → Sink

7.3 Common Patterns Across All Data Flows

Pattern	Detail
Date surrogate keys	toInteger(toString(DateColumn, 'yyyyMMdd')) — converts date to integer matching dim_date.date_sk
Sink table action	Truncate table — ensures fresh load on every run, no duplicate accumulation
Broadcast hint	Optimize → Broadcast → Fixed → Right — prevents DF-JOIN-010 errors on lookup joins
batch_id	'ADF_DF_RUN' — identifies rows loaded by Data Flow vs Stored Procedure
dwh_load_date	currentTimestamp() — records exact load time for each row

8. Data Validation & Error Handling

A dedicated validation stored procedure runs after CSV data is loaded into staging but before any DWH loading begins. This catches bad data early before it contaminates the DWH.

8.1 Validation Checks

#	Check	What it looks for	Tables checked
1	NULL primary keys	Any row where the main ID column is NULL	All 11 stg_ tables
2	Duplicate IDs	Same ID appearing more than once	stg_clients, stg_trades, stg_accounts
3	Referential integrity	Trades referencing non-existent accounts	stg_trades, stg_accounts
4	Invalid values	Zero-amount trades, negative holdings, future trade dates	stg_trades, stg_holdings
5	Row count threshold	Staging tables with fewer than 100 rows	stg_clients, stg_trades, stg_holdings

8.2 Duplicate TradeIDs — Real Issue Found

When we first ran validation, it flagged 400 duplicate TradeIDs in stg_trades. These were present in the original Trades.csv — a source data quality issue, not a pipeline problem. We removed them using ROW_NUMBER() and stg_trades went from 100,400 to 100,000 rows.

8.3 Error Logging

All validation failures are written to pipeline_error_log. All successful runs are written to pipeline_audit_log. This gives a full history of every pipeline run for debugging.

9. ADF Pipelines

9.1 Linked Services

Name	Type	Connects to
LS_BlobStorage	Azure Blob Storage	finwealthsa — where CSV files are stored
LS_AzureSQL	Azure SQL Database	finwealth-sql-server / FinWealthDW

9.2 Datasets

22 datasets created — 11 Blob datasets (one per CSV) and 11 SQL datasets (one per staging table).

9.3 PL_Copy_CSV_To_Staging_To_DWH Pipeline

Main pipeline that copies CSV files to staging and loads dimension tables via SPs.

Stage	Activities	What happens
Stage 1	11 Copy Activities	All 11 CSV files copied from Blob to staging in parallel. TRUNCATE pre-copy script ensures fresh load.
Wait1	Wait Activity	Waits for all 11 Copy Activities before dimension loading starts.
Stage 2	5 SP Activities	5 dimension SPs run in parallel — Dim_Advisor, Dim_Client, Dim_Account, Dim_Security, Dim_Benchmark.
Wait2	Wait Activity	Waits for all 5 dimension SPs before fact loading.
Stage 3	5 SP Activities (validation)	usp_validate_staging runs. Results written to audit/error log tables.

9.4 PL_Load_Fact — Data Flow Pipeline

Dedicated pipeline running all 5 Mapping Data Flows in parallel. Each Data Flow truncates its target fact table before inserting, ensuring idempotent loads.

Activity	Data Flow	Status	Duration
DF_Cashflows	DF_Load_Fact_Cashflows	Succeeded	4m 8s
DF_Performance	DF_Load_Fact_Performance	Succeeded	6m 12s
DF_Fees	DF_Load_Fact_Fees	Succeeded	3m 31s
DF_Holdings	DF_Load_Fact_Holdings	Succeeded	6m 39s
DF_Trades	DF_Load_Fact_Trades	Succeeded	5m 18s

9.5 PL_Validation Staging Pipeline

Separate pipeline running `usp_validate_staging` independently without triggering a full reload. Results written to `pipeline_audit_log` and `pipeline_error_log`.

9.5 PL_Master Pipeline

PL_Master is the single entry point for the entire FinWealth data pipeline. It orchestrates three child pipelines in strict sequence using Execute Pipeline activities connected by success conditions (green arrows). First, PL_Copy_CSV_To_Staging copies all 11 CSV files from Azure Blob Storage into the staging layer. On success, PL_Validation runs `usp_validate_staging` to perform NULL checks, duplicate detection, referential integrity checks, and row count threshold validation — results are written to `pipeline_audit_log` and `pipeline_error_log`. Only if validation passes does PL_Load_Fact execute, running all 5 Mapping Data Flows in parallel to load the fact tables. This design ensures bad data can never reach the data warehouse — if any step fails, the pipeline stops and the error is logged automatically.

10. Fact Table DDL Scripts

10.1 Fact_Trades

```
CREATE TABLE Fact_Trades (trade_sk INT IDENTITY(1,1) PRIMARY KEY,  
trade_id VARCHAR(50),  
account_sk INT,  
security_sk INT,  
trade_date_sk INT,  
settle_date_sk INT,  
side VARCHAR(10),  
price DECIMAL(18,4),  
quantity DECIMAL(18,4),  
trade_amount DECIMAL(18,2),  
gross_amount DECIMAL(18,2),  
TradeCategory NVARCHAR(50),  
TradeSize NVARCHAR(50),  
dwh_load_date DATETIME,  
batch_id VARCHAR(50))
```

10.2 Fact_Cashflows

```
CREATE TABLE Fact_Cashflows (cashflow_sk INT IDENTITY(1,1) PRIMARY KEY,  
cashflow_id VARCHAR(50),  
account_sk INT,  
cashflow_date_sk INT,  
cashflow_type VARCHAR(50),  
amount DECIMAL(18,2),  
FlowDirection NVARCHAR(50),  
dwh_load_date DATETIME,  
batch_id VARCHAR(50))
```

10.3 fact_fees

```
CREATE TABLE fact_fees (fee_sk INT IDENTITY(1,1) PRIMARY KEY,  
fee_id VARCHAR(50),  
account_sk INT,  
fee_date_sk INT,  
fee_type VARCHAR(50),  
amount DECIMAL(18,2),  
dwh_load_date DATETIME,  
batch_id VARCHAR(50))
```

10.4 Fact_Performance

```
CREATE TABLE Fact_Performance (performance_sk INT IDENTITY(1,1) PRIMARY KEY,  
account_sk INT,  
benchmark_sk INT,  
month_start_date_sk INT,  
return_pct DECIMAL(18,6),  
benchmark_return_pct DECIMAL(18,6),  
excess_return_pct DECIMAL(18,6),  
cumulative_return_pct DECIMAL(18,6),  
PerformanceBand NVARCHAR(50),  
dwh_load_date DATETIME,  
batch_id VARCHAR(50))
```

10.5 fact_holdings

```
CREATE TABLE fact_holdings (holding_sk INT IDENTITY(1,1) PRIMARY KEY,  
account_sk INT,  
security_sk INT,  
holding_date_sk INT,  
quantity DECIMAL(18,4),  
market_value DECIMAL(18,2),  
close_price DECIMAL(18,4),  
dwh_load_date DATETIME,  
batch_id VARCHAR(50))
```

11. Final DWH Row Counts

Table	Row Count	Notes
dim_date	5,844	Pre-populated date spine from 2015 to 2030
Dim_Advisor	350	All advisors loaded with ExperienceBand
Dim_Client	8,000	All clients with ClientTier and TenureYears
Dim_Account	12,000	All accounts with CurrencyGroup
Dim_Security	3,000	All securities with AssetClassGroup
Dim_Benchmark	6	6 distinct benchmark codes
fact_holdings	96,027	Daily holdings with market values
Fact_Trades	100,000	After dedup — 400 duplicates removed from source
Fact_Cashflows	30,000	Deposits, withdrawals, dividends, interest
fact_fees	18,000	Advisory, management, and transaction fees
Fact_Performance	110,000	Monthly returns with benchmark comparison

12. Power BI Analytical Views

13 pre-joined SQL views created so Power BI connects to ready-to-use datasets without slow multi-table joins.

12.1 AUM Summary Views

View	What it provides
vw_aum_summary	Full AUM detail by client, advisor, security, date
vw_aum_by_client_tier	AUM rolled up by ClientTier — Platinum vs Gold vs Silver vs Standard
vw_aum_by_asset_class	AUM by asset class and AssetClassGroup — product mix analysis
vw_aum_by_region	AUM grouped by advisor region — geographic breakdown
vw_monthly_aum_trend	AUM by year, quarter, month — for trend line charts
vw_product_mix	Detailed product mix with percentage of total AUM per asset class

12.2 Commission & Fee Views

View	What it provides
vw_commission_summary_by_advisor	Commission totals per advisor with fee type breakdown
vw_fee_summary	Fees by advisor, account type, fee type, and date

12.3 Performance Views

View	What it provides
vw_performance_dashboard	Monthly returns vs benchmark by account, advisor, client
vw_performance_by_risk	Average returns grouped by client risk profile

12.4 Supporting Views

View	What it provides
vw_advisor_performance	AUM, trade volumes, and fees per advisor in one view
vw_advisor_experience	Advisor count and average AUM by experience band and region
vw_clients_by_country	Client count by country, segment, and tier
vw_trade_summary	Trade volumes by side (BUY/SELL), category, and size

13. Assignment Objectives — Completion Status

#	Objective	Status	What we delivered
1	Design a Star Schema for wealth analytics	Done	6 dim tables + 5 fact tables with surrogate keys
2	Develop ETL Pipelines in ADF	Done	PL_01 (copy + SP load), PL_02_DF_Fact_Load (Data Flows), PL_02 (validation)
3	Implement Data Validation and Error Handling	Done	usp_validate_staging with 5 check categories + audit/error log tables
4	Optimize SQL transformations for aggregation	Done	13 analytical views + aggregation queries
5	Build fact and dimension tables with surrogate keys	Done	All 11 DWH tables use IDENTITY surrogate keys
6	Produce analytical datasets for Power BI	Done	AUM, Commission, Performance views created

14. Key Decisions & Lessons Learned

Hybrid Approach — SPs for Dims, Data Flows for Facts

We used Stored Procedures for dimension loading (SCD Type 1 merge, surrogate keys, derived columns) and ADF Mapping Data Flows for fact loading (high-volume joins, Spark parallel processing). This gives the best of both worlds — SQL testability for business logic and Spark performance for large datasets.

Duplicate TradeIDs in Source Data

Trades.csv had 400 duplicate TradeIDs present in the original source file. Our validation caught this before it reached the DWH. We removed duplicates using ROW_NUMBER() and documented the issue. This is exactly what a validation layer is designed to catch.

TRUNCATE in Pre-Copy Script

Initially staging tables accumulated double rows when pipelines ran twice. Adding TRUNCATE TABLE pre-copy scripts to ADF Copy Activity Sink settings fixed this — re-running always produces a clean fresh load.

Broadcast Hint for Lookup Joins

Data Flow lookups against dimension tables threw DF-JOIN-010 errors due to non-equality join conditions (toString casting). Setting Optimize → Broadcast → Fixed → Right on all Lookup steps resolved this.

Date Surrogate Key Pattern

Instead of adding separate Lookup steps to match dates against dim_date, we used toInteger(toString(DateColumn, 'yyyyMMdd')) in Derived Column to directly generate the integer surrogate key. This simplified the canvas significantly.